

When to Ask, What to Say, and When Listening Hurts: Communication Between Asymmetric LLM Agents

Anonymous ACL submission

Abstract

When a small LLM agent can consult a stronger model that is only allowed to advise, everything the stronger model contributes has to pass through language. The small *executor* decides when to ask, the *advisor* decides what to say, and the executor decides whether to follow the advice. We study this exchange as a communication problem, using five executor–advisor pairs from three model families (GPT, Qwen, gpt-oss) and three pairs that mix families, on GAIA and HotpotQA. *When to ask*: changing the rule that opens the channel can affect accuracy more than replacing the executor with a larger model, and the best rule depends on the type of failure. On HotpotQA, where failures are mostly silent reasoning errors, letting the executor ask for help works best in every pair, seed, and advisor we tested. On GAIA, where failures are mostly visible tool errors, a trigger that combines an error monitor with the executor’s confidence works best for the GPT executors and ties with the executor’s own request for the open-weight ones. *What to say*: we split advice into three communicative acts: DIAGNOSE (what went wrong), AVOID (what not to do), and NEXT (what to do now). Each act contributes, and the full message beats free-form advice in all eight pair–benchmark settings. The gain requires a stronger speaker: the same message written by the executor to itself does not help. The acts matter more than their surface form: writing them as compact labelled fields instead of full sentences changes accuracy by at most two points while using 40% of the tokens, and both versions outperform sending the next action or the advisor’s own answer. *When listening hurts*: advice turns a correct answer into a wrong one on 2–6% of tasks. Harm is higher without the AVOID act, with free-form text, and when the advisor hands over actions instead of explaining. Swapping advisors across model families shows that a stronger advisor mainly adds help, while the number of broken tasks barely changes.

1 Introduction

LLM agents solve multi-step tasks by reasoning, calling tools, and reading the results (Yao et al., 2023; Schick et al., 2023; Yang et al., 2026; Ruan et al., 2023). They fail in well-documented ways: they misuse tools, pass malformed parameters, misread observations, and drift in their reasoning (Srikumar et al., 2025; Sun et al., 2024; Qin et al., 2023; Guo et al., 2025; Huang et al., 2024b; Li, 2025; Masterman et al., 2024). A common way to make agents both capable and affordable is to let a small model do most of the work and consult a stronger model only when needed (Ding et al., 2024; Shen et al., 2024; Wang et al., 2025a). In the *advisor* version of this design (Anthropic, 2026), the stronger model never takes over. It can only give advice.

This restriction turns a routing problem into a communication problem. A router picks which model answers a query (Chen et al., 2023; Ong et al., 2025). An advisor cannot answer or call tools, so everything it contributes must be put into a message, and the small model decides what to do with that message. This is the situation that dialogue research studies: a speaker and a listener with unequal knowledge, a decision about when to talk, a choice of what to say, and a risk of being misunderstood (Clark and Brennan, 1991; Inan et al., 2026).

We do not propose the advisor architecture. We use it as a controlled setting in which only the communication changes, and ask three questions.

When should the executor ask? To ask for help, the executor must notice that something is wrong. Models can report their confidence, but these reports are poorly calibrated (Kadavath et al., 2022; Xiong et al., 2024), and many agent failures are *silent*: nothing visibly breaks (Sun et al., 2024). It is not obvious whether an external error monitor or the executor itself should decide when to ask.

What should the advisor say? Feedback between models is usually free-form text (Madaan et al., 2023; Gou et al., 2024). An advisor could also state a diagnosis, forbid a step, give an instruction, hand over the next action, or simply give its own answer. Each option transmits different information and trusts the listener to a different degree.

When does listening hurt? LLMs tend to accept confident input, even when it contradicts evidence they already have (Sharma et al., 2024; Xie et al., 2024; Wang et al., 2023). An executor that is on the right track can therefore be led away from a correct answer by advice that sounds reasonable but is wrong.

We answer these questions with five executor–advisor pairs from three model families: three OpenAI executors with a GPT-5.4 advisor, Qwen3.5-9B with a Qwen3.6-27B advisor, and gpt-oss-20b with a gpt-oss-120b advisor. We also swap advisors across families (e.g., Qwen3.5-9B advised by GPT-5.4) and repeat the main comparison with three random seeds. We use two benchmarks with different kinds of failure. On GAIA (Mialon et al., 2023), most failures are visible tool errors. On HotpotQA (Yang et al., 2018), most failures are silent errors in combining facts. All systems share the same agent code, prompts, and budgets. Figure 2 summarises the main results.

Contributions.

- We describe weak–strong agent collaboration as **two-way communication** with three design choices: when to open the channel, what the message contains, and whether the listener follows it.
- We show that **the best moment to ask depends on where the evidence of failure is**. When failures are silent, the executor’s own request is best in every pair, seed, and advisor we tested. When failures are visible, a combined error-and-confidence trigger is best for the GPT executors.
- We decompose advice into **three communicative acts** (DIAGNOSE / AVOID / NEXT) and show that each contributes across all pairs and both benchmarks. Comparing seven ways of sending advice, we find that the gain requires a stronger speaker, depends more on which acts are sent than on whether they are written as compact fields or full sentences, and is

larger when the advisor explains than when it acts or answers.

- We measure **when advice hurts** across eight executor–advisor combinations. Harm is rare but real, and it is higher with unstructured messages, without the AVOID act, and with action hand-offs. Swapping advisors shows that a stronger advisor mainly adds help, while harm stays nearly constant.

2 Related Work

Agent communication. LLM agents coordinate through natural language in role-play (Li et al., 2023), conversation frameworks (Wu et al., 2023; Hong et al., 2024), debate (Du et al., 2024; Liang et al., 2024), and layered aggregation (Wang et al., 2024); see Guo et al. (2024) for a survey. Many multi-agent failures happen in the communication itself: information is ignored or withheld (Cemri et al., 2025), errors pass through message hand-offs (Lin et al., 2026), and agents can communicate a lot yet still fail to compute the right answer together (Zhang et al., 2026). Inan et al. (2026) argue that dialogue should be part of an agent’s planning loop. Most of this work studies agents of similar capability. We study a pair with *unequal* capability, where the stronger agent can only talk.

When to communicate. Deciding when to ask for help has been studied for robots (Ren et al., 2023), for clarification questions to human users (Dong et al., 2026), and for the timing of clarification in long tasks (Gulati et al., 2026). ADaPT decomposes a task further only when the executor fails (Prasad et al., 2024). Whether a model’s uncertainty is worth sending depends on how well calibrated it is (Kadavath et al., 2022; Tian et al., 2023; Xiong et al., 2024) and on how it is expressed in language (Mielke et al., 2022; Zhou et al., 2024). Learning to defer trains a model to hand a decision to a more reliable expert (Madras et al., 2018; Mozannar and Sontag, 2020). Cost-aware cascades and routers choose which model answers a query (Chen et al., 2023; Ong et al., 2025; Aggarwal et al., 2025; Wang et al., 2025b; Ding et al., 2024; Yue et al., 2024; Li et al., 2025; Lakha et al., 2025), often with fixed heuristics (Wang et al., 2026). In deferral and routing, the stronger model’s *decision replaces* the weaker model’s decision. In our setting, the stronger model sends a *message* that the weaker model has to interpret. This is why

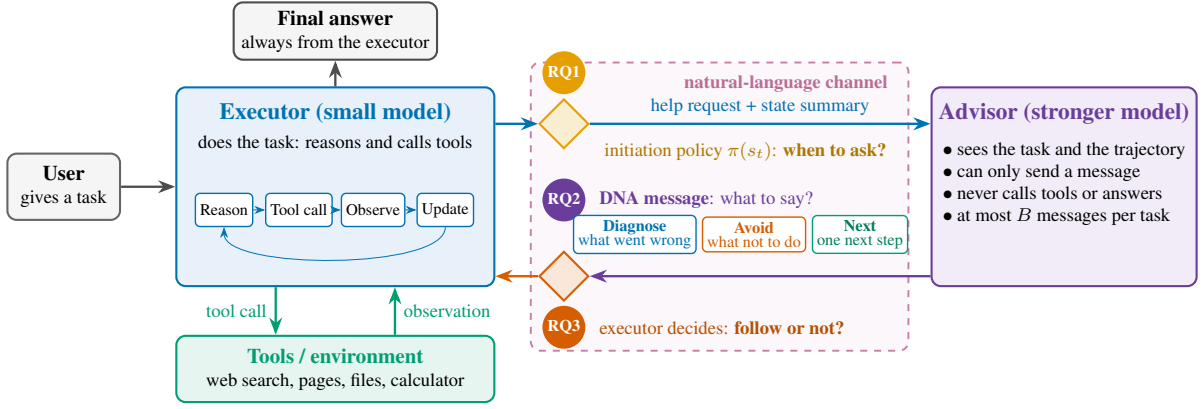


Figure 1: The Advisor Pattern as communication between two agents. The executor reasons and calls tools. An initiation policy decides when to open a natural-language channel to a stronger advisor and sends a help request with a summary of the current state. The advisor replies with a short DNA message (DIAGNOSE / AVOID / NEXT), and the executor decides whether to act on it. The advisor never calls tools and never gives the final answer. The three decision points correspond to our research questions: when to ask (RQ1), what to say (RQ2), and whether to follow (RQ3).

harm can come from how advice is understood, a failure that deferral models do not capture.

What to communicate. Natural-language feedback is used for self-refinement (Madaan et al., 2023; Shinn et al., 2023) and for tool-grounded critique (Gou et al., 2024; Huang et al., 2025). Self-correction without an external signal is unreliable (Huang et al., 2024a; Kamoi et al., 2024), and small models benefit from a stronger verifier (Zhang et al., 2024). Saha et al. (2023) show that a teacher model can improve a weaker student with explanations under an intervention budget. We move this idea to multi-step tool use, where advice arrives in the middle of a task. On message form, PACT limits messages between agents to action, state, and result fields and finds that free-form text is costly and rarely best (Huang et al., 2026). Other work models dialogue as a choice among communicative intents (Ye et al., 2026), adapts a strong model’s language to weaker partners (West et al., 2026), or replaces language with embeddings or hidden states (Pham et al., 2024; Du et al., 2026; Foerster et al., 2016; Lazaridou and Baroni, 2020). DNA is closest to PACT, but it is corrective rather than a hand-off between peers, and we test each act separately. Hidden states are not available from API models, so we compare natural language with compact, action-level, and answer-level messages instead.

Grounding and reliance. Communication works when both sides share an understanding

of the relevant facts (Clark and Brennan, 1991), and a cooperative speaker says as much as needed and no more (Grice, 1975). LLM listeners are sycophantic (Sharma et al., 2024; Perez et al., 2023), can be argued out of correct beliefs (Wang et al., 2023), prefer coherent outside evidence over their own (Xie et al., 2024), and model their partners imperfectly (Sap et al., 2022). In human–AI teams, advice can make the team worse when people cannot judge it (Bansal et al., 2021). That following advice can hurt is therefore known. We do not identify message-level causal mechanisms; instead, we measure how the timing, form, and source of advice change the balance between help and harm in agent–agent communication.

3 The Advisor Pattern as Communication

3.1 Setting

Let x be a task and s_t the executor’s state at step t (the task, the reasoning so far, and all tool outputs). A small executor M_e proposes an action $a_t = M_e(s_t)$: either a tool call or a final answer. A stronger advisor M_a can be consulted through a channel with two directions:

- **Executor → advisor.** An *initiation policy* $\pi(s_t) \in \{0, 1\}$ decides whether to open the channel. If it opens, the advisor receives the task, the trajectory so far (reasoning, tool calls, and observations), any error signals, the current candidate answer, and, if the executor asked, its request.

- **Advisor** $\rightarrow$ **executor**. A message format μ fixes what the advisor may say. The advisor’s message $g_t = M_a(c_t; \mu)$ is added to the executor’s context, and the executor decides how to act on it.

The design problem is to choose both *when* (π) and *what* (μ). Accuracy summarises the result, but we also count help and harm separately (§4), because a policy that rescues many tasks while breaking some may be unacceptable where overwriting a correct answer is costly.

Three constraints make the advisor a communicator, not a solver. (i) *Budget*: at most B messages per task. (ii) *No actions*: only the executor calls tools or gives the final answer. (iii) *No oracle*: the advisor does not know whether the executor is already on the right track. Constraint (iii) is what makes harmful advice possible (§7). Algorithm 1 in Appendix A gives the full loop.

3.2 When to talk: initiation policies

The policies differ in *who holds the evidence* that help is needed.

- **Monitor** (external). Fixed rules open the channel after a visible breakdown: a tool error, malformed output, or a repeated identical call (Huang et al., 2025).
- **Report** (executor’s confidence). After each step, the executor reports a confidence between 0 and 1. The channel opens when it falls below a threshold τ .
- **Self-initiated** (executor’s request). The executor can send an explicit help request instead of an action.
- **Hybrid**. The channel opens on a visible error or low confidence: $\pi(s_t) = 1$ if $\text{error}(s_t) \vee \text{conf}(s_t) < \tau$.
- **Controls**. *Random* opens the channel at random with the same budget. *No channel* runs the executor alone.

A monitor sees only failures that break something visibly. Confidence reports and help requests expose the executor’s own view of its progress, but they are only as good as its self-knowledge.

3.3 What to say: DNA as three communicative acts

Corrective advice has to do three things: explain what went wrong, keep the listener away from

known mistakes, and propose a repair. DNA makes each one a separate, labelled act (Searle, 1976):

- **DIAGNOSE** (a statement): the likely cause of the problem, such as a wrong tool, a bad parameter, or a reasoning error (Sun et al., 2024). It gives both agents the same account of the problem.
- **AVOID** (a prohibition): what *not* to do next, such as retrying a failing call or re-checking facts that are already established.
- **NEXT** (an instruction): one concrete next step.

We do not claim that these three acts cover all corrective communication. They are one decomposition whose parts we can test separately (§6). In line with the cooperative principle (Grice, 1975), messages are short, focused on the current state, and leave execution to the executor. By default, each act is a short labelled field (about 70 tokens per message in total); §6.2 compares this with writing the same acts as full sentences. Appendix Figure 4 shows an example.

4 Experimental Setup

Benchmarks with two kinds of failure. We chose two benchmarks not for breadth but because they create two contrasting communication settings: one where the need for help is visible from outside, and one where it is not. **GAIA** (Mialon et al., 2023): the full validation set (165 tasks: 53 Level 1, 86 Level 2, 26 Level 3). Its public answers allow a task-by-task comparison between runs. Failures are mostly *visible*: bad URLs, malformed parameters, parsing errors. **HotpotQA** (Yang et al., 2018): 300 questions (150 bridge, 150 comparison) with one `wiki_search` tool that returns the opening sections of the top three articles. Failures are mostly *silent*: a missing linking entity, a wrong combination of facts, or an answer given too early. Nothing visibly breaks; the trajectory simply heads toward a wrong answer.

Executor–advisor pairs. We use five pairs, named by their executor. **Nano**, **Mini**, and **4.1** are GPT-5.4 Nano, GPT-4.1 Mini, and GPT-4.1, all advised by GPT-5.4. **Qwen** is Qwen3.5-9B advised by Qwen3.6-27B. **OSS** is gpt-oss-20b advised by gpt-oss-120b. The open-weight models are served locally with vLLM. Each pair has its own reference, *advisor alone*: the advisor model

running the whole task by itself. To separate the advisor from the model family, we also run three cross-family pairs: Qwen3.5-9B and gpt-oss-20b advised by GPT-5.4, and GPT-5.4 Nano advised by Qwen3.6-27B.

Settings. All systems share the same agent code, prompts, and tools; advised runs differ from executor-only runs *only* through the channel. Each task allows at most 16 steps, 12 tool calls, and $B = 2$ advisor messages. The hybrid policy uses $\tau = 0.75$. GPT-4.1 models run at temperature 0; GPT-5.4 models run with low reasoning effort. Each configuration is run with seed 42. For the main comparison (no channel, hybrid, self-initiated), we add seeds 43 and 44 for Nano, Qwen, and OSS. Appendix A gives further details and prices.

Measures. **Accuracy** is normalised exact match. To see what advice does to individual tasks, we compare each advised run with the executor-alone run on the same tasks. A **help** event is a task that is wrong alone and right with advice. A **harm** event is a task that is right alone and wrong with advice. Advised accuracy always equals executor-alone accuracy plus (help – harm) divided by the number of tasks. We also report **tokens per message** and **recovery efficiency** (accuracy points gained per advisor message).

Statistics. We test each advised run against the executor-alone run with an exact McNemar test on its help and harm counts (Agresti, 2012). For configurations run once, a single difference between two advised policies is reliable only above roughly 5 points on GAIA and 4 points on HotpotQA. We therefore draw conclusions from differences that point the same way across pairs and benchmarks, and we report how often they do.

5 RQ1: When Should the Executor Ask?

When the channel opens matters as much as who is listening. Table 1 shows accuracy for each initiation policy. For Nano on GAIA, the policy alone moves accuracy from 18% (random) to 48% (hybrid). Replacing Nano with the larger GPT-4.1 as the executor, without any advice, moves accuracy only from 20% to 31%. Having access to a stronger model is not enough: random initiation, with the same message budget, gains between –2 and +5 points on GAIA and between +2 and +5 on HotpotQA. For the best single-signal policy and

for the hybrid, the gain over the executor alone is significant in every pair on GAIA (exact McNemar test, $p \leq 0.003$; Appendix Table 11).

There is no single best initiator: it depends on where the evidence is. Figure 2a compares the hybrid trigger with self-initiated requests. With seed 42, the hybrid is better on GAIA and self-initiated requests are better on HotpotQA in all five pairs. Two more seeds (Appendix Table 7) show which half is robust; accuracy moves by at most 2 points between seeds. On HotpotQA, self-initiated requests win in every seed for every pair. On GAIA, the hybrid wins every seed for Nano (by 6–8 points), but for Qwen and OSS the two stay within 2 points and the winner changes between seeds. The three cross-family pairs agree (Appendix Table 8): self-initiated requests win on HotpotQA in all three, while GAIA gives +1, 0, and –1. In total, self-initiated requests win on HotpotQA in 14 of 14 settings, while the hybrid’s advantage on GAIA is clear only for the GPT executors.

The monitor explains the pattern. On its own, it is the weakest, or tied for weakest, of the three single signals (monitor, report, self-initiated) in all ten settings, because it fires only after a visible breakdown. On GAIA, many failures are visible, and adding the monitor to the confidence report still helps: the hybrid matches or beats the report policy in all five pairs. On HotpotQA, a trajectory that misses a linking entity looks healthy from outside, and only the executor’s own sense of being stuck carries the signal. *When failures are silent, the executor’s own request is the right initiator for every executor we tested.* This agrees with findings that the value of asking depends on what information is missing (Gulati et al., 2026).

Advised pairs match the advisor alone on HotpotQA, but not on GAIA. On GAIA, the best advised configuration reaches 78–91% of the advisor’s own accuracy (e.g., 48% vs. 55% for Nano). On HotpotQA, it matches or exceeds the advisor alone in every pair (+1 to +11 points; the Qwen margin is within noise).

The channel pays off across a capability gap. For Nano and Mini, advised runs cost 67–81% less than running GPT-5.4 alone; GPT-4.1 is priced close to its advisor, so its advised runs are not cheaper (Appendix Table 5).

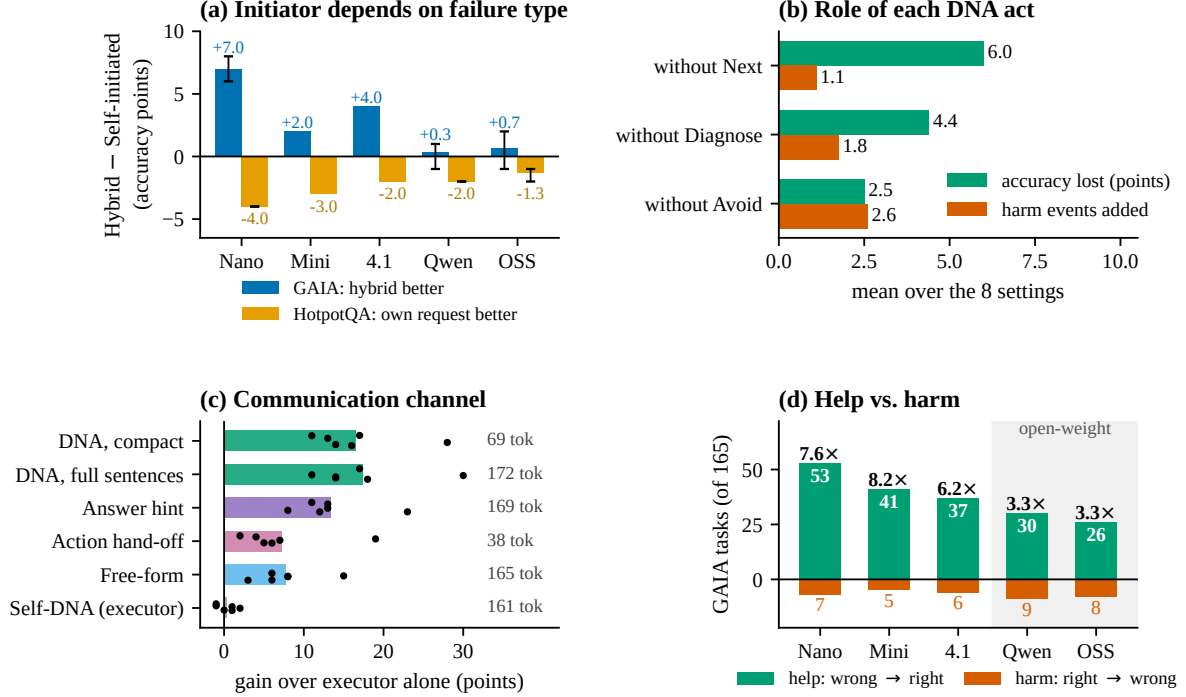


Figure 2: Main results at a glance. **(a)** Hybrid minus self-initiated accuracy per pair; Nano, Qwen, and OSS show the mean over three seeds (whiskers: lowest and highest), Mini and 4.1 a single seed. **(b)** Accuracy lost and harm events added when one DNA act is removed, averaged over the eight settings of Table 12. **(c)** Gain over the executor alone for each channel in Table 2 (bar = mean, dots = the six settings; tokens per message for Nano). **(d)** GAIA tasks rescued (up) and broken (down) by the hybrid policy; the number above each bar is the help/harm ratio (Appendix Table 11).

6 RQ2: What Should the Advisor Say?

RQ1 decides when the channel opens. We now hold that decision fixed and ask what information should cross it.

6.1 Which communicative acts matter?

We fix the initiation policy (hybrid) and change only the message. We test all seven non-empty subsets of {DIAGNOSE, AVOID, NEXT} and free-form advice without labelled acts (Figure 2b; full results in Appendix Table 12).

Every act contributes, and the full message is best in all eight settings. Full DNA has the highest accuracy in all eight columns. Removing any single act lowers accuracy in all eight columns.

NEXT matters most, then DIAGNOSE, and AVOID mainly limits harm. Averaged over pairs, removing NEXT costs the most (6.8 points on GAIA, 4.7 on HotpotQA), removing DIAGNOSE costs less (4.8 and 3.7), and removing AVOID costs the least (2.4 and 2.7). NEXT alone is the strongest single act in all eight columns. That DIAGNOSE

still adds to NEXT suggests that a shared account of the problem helps the executor apply the instruction that follows. AVOID plays a different role. Without it, the number of harm events rises in all eight columns (e.g., from 7 to 10 for Nano on GAIA). An explicit prohibition keeps the executor from undoing work that was already correct.

Structure beats free-form text. Free-form advice is less accurate than full DNA in all eight columns, by 7–13 points on GAIA and 6–11 on HotpotQA, and it causes more harm in all eight columns. In the logs, free-form advice tends to trigger broad rewrites of the trajectory, while DNA leads to targeted repairs.

6.2 Who speaks, and through which channel?

The ablation leaves three questions open. Is the gain due to the stronger *speaker*, or would the format help even if the executor wrote it itself (Shinn et al., 2023; Madaan et al., 2023)? Does the *surface form* matter, or only the acts? And is explaining better than simply *handing over* an action or an answer (Huang et al., 2026)? Table 2 and Figure 2c

Initiation policy	GAIA (165 tasks; failures mostly visible)					HotpotQA (300 tasks; failures mostly silent)				
	Nano	Mini	4.1	Qwen	OSS	Nano	Mini	4.1	Qwen	OSS
No channel (executor alone)	20	26	31	23	27	61	69	73	61	64
Random	18	31	36	27	30	65	71	78	64	67
Monitor	39	42	43	31	31	79	77	84	70	72
Report ($\tau=.75$)	45	46	48	35	38	80	81	84	75	80
Hybrid (monitor or report)	48	48	50	36	38	78	82	87	75	80
Self-initiated	40	46	46	35	37	82	85	89	77	81
Advisor alone	55	55	55	44	49	78	78	78	76	76

Table 1: Accuracy (%) by initiation policy. Columns are executor–advisor pairs: Nano, Mini, 4.1 = GPT-5.4 Nano, GPT-4.1 Mini, GPT-4.1 advised by GPT-5.4; Qwen = Qwen3.5-9B advised by Qwen3.6-27B; OSS = gpt-oss-20b advised by gpt-oss-120b. Bold: best advised policy per column. “Advisor alone” is the pair’s advisor solving the task by itself (GPT-5.4 for the first three pairs). Seed 42; repeated seeds are in Appendix Table 7. Other thresholds, cost, and latency are in Appendix B.

compare seven channels with the same trigger (hybrid) and budget: **Self-DNA**: the executor writes the DNA message to itself. **Free-form** and **free-form, length-matched**: advice without labelled acts; the second is limited to the length of a full-sentence DNA message. **DNA, compact**: the default, three short labelled fields. **DNA, full sentences**: the same three acts written as normal prose. **Action hand-off**: the advisor sends only the exact next tool call, which is executed. Here the advisor effectively acts, breaking constraint (ii). **Answer hint**: the advisor states the answer it believes is correct, and the executor may use it.

The gain requires a stronger speaker. When the executor writes the DNA message to itself, accuracy stays within 2 points of the executor alone in all six settings, and help and harm counts stay small (at most 6). The difference is significant in only one setting (HotpotQA, Qwen: 6 help and 0 harm, $p = 0.03$), and not after correcting for the six comparisons. The labelled format does not create useful information by itself. The gain therefore cannot be explained by the DNA scaffold acting as self-reflection; it depends on information supplied by the stronger speaker.

The acts matter more than their surface form. Writing the same three acts as full sentences changes accuracy by at most 2 points in all six settings, while making messages 2.5 times longer (172 vs. 69 tokens for Nano). Free-form advice stays 6–13 points below compact DNA in all six settings, and limiting its length to that of full-sentence DNA changes it by at most 1 point, so the gap is not caused by message length. Within what we tested, the communicative acts carry the benefit, and fluent

prose is not needed to deliver them. The messages remain language, but their function matters more than their length or style. One exception is worth noting: for Nano on GAIA, full sentences produced both more help (66 vs. 53) and more harm (16 vs. 7) than the compact form, as if longer messages push the executor harder in both directions.

Explaining beats acting or answering. Handing over the next action is 6–13 points below compact DNA in all six settings. On HotpotQA it is even below free-form advice, and it causes the most harm of any advisor channel in all three pairs (11, 12, and 15 events). A tool call repairs one step, but it does not tell the executor what went wrong or what to avoid. Giving the advisor’s own answer is also below compact DNA in all six settings (2–5 points). Because the advisor sees only the partial trajectory, its direct answer may rest on incomplete evidence; structured advice instead leaves the executor to gather that evidence and decide for itself. In both cases, the channel that explains outperforms the channel that bypasses the executor’s reasoning.

7 RQ3: When Does Listening Hurt?

RQ2 shows which messages improve accuracy. We now ask what the same messages can break.

Harm is rare, but real. Figure 2d and Appendix Table 11 count, for each pair, the tasks that advice rescued and the tasks it broke. Advice rescues 16–32% of GAIA tasks and breaks 2–6%. Some transitions may come from run-to-run variation, so harm counts are an upper bound.

Swapping the advisor: help changes, harm barely does. The open-weight pairs have lower

Channel	Speaker	Nano	GAIA Qwen	OSS	Nano	HotpotQA Qwen	OSS	Tokens per message (Nano)
No channel	–	20	23	27	61	61	64	–
Self-DNA	executor	20 (0/0)	24 (2/0)	26 (1/3)	62 (2/0)	63 (6/0)	63 (0/3)	161
Free-form	advisor	35 (36/11)	29 (24/14)	30 (16/11)	67 (28/10)	69 (35/11)	72 (34/10)	165
Free-form, length-matched	advisor	34 (34/11)	29 (25/15)	29 (16/12)	67 (28/10)	69 (35/11)	71 (32/11)	166
DNA, compact (default)	advisor	48 (53/7)	36 (30/9)	38 (26/8)	78 (58/7)	75 (51/9)	80 (57/9)	69
DNA, full sentences	advisor	50 (66/16)	37 (32/9)	38 (25/7)	79 (59/6)	75 (51/9)	81 (60/9)	172
Action hand-off	advisor	39 (47/16)	30 (23/11)	29 (16/13)	65 (23/11)	67 (30/12)	69 (30/15)	38
Answer hint	advisor	43 (48/10)	34 (27/9)	35 (22/9)	74 (47/8)	73 (45/9)	77 (49/10)	169
Advisor alone	–	55	44	49	78	76	76	–

Table 2: Speaker and channel comparison (hybrid initiation, same budget). Cells show accuracy (%) with (help / harm) counts against the executor alone. Bold: best advised channel per column.

Executor	Advisor	Alone	GAIA	HotpotQA
Qwen3.5-9B	none	–	23	61
	Qwen3.6-27B	44	36 (30/9)	77 (54/6)
	GPT-5.4 [†]	55	39 (35/9)	81 (67/7)
gpt-oss-20b	none	–	27	64
	gpt-oss-120b	49	38 (26/8)	81 (58/7)
	GPT-5.4 [†]	55	40 (29/8)	82 (62/8)
GPT-5.4 Nano	none	–	20	61
	GPT-5.4	55	48 (53/7)	82 (67/4)
	Qwen3.6-27B [†]	44	43 (47/9)	77 (55/7)

Table 3: Swapping the advisor while keeping the executor fixed. Cells: accuracy (%) with (help / harm) against the executor alone. GAIA uses the hybrid trigger; HotpotQA uses self-initiated requests. “Alone” is the advisor’s own GAIA accuracy. [†]Cross-family pair.

help/harm ratios (about 3, vs. 6–9), but their advisors differ in both strength and family. To separate the two, we keep the executor fixed and change only the advisor (Table 3). The stronger advisor gives higher accuracy in all six comparisons: GPT-5.4 raises Qwen3.5-9B and gpt-oss-20b on both benchmarks, and the weaker Qwen3.6-27B lowers Nano. Advice also works across model families: all three cross-family pairs improve significantly over their executor alone (exact McNemar test, $p < 0.001$). The change comes almost entirely from help. Across the six comparisons, help changes by 3–13 tasks, while harm changes by at most 3. On GAIA, harm stays between 7 and 9 tasks in every row of Table 3, whichever advisor is used. The lower ratio of the open-weight pairs is therefore mainly a difference in how much the executor gains from advice, not in how often advice breaks correct work.

The form of the message changes how much harm it does. Harm rises without AVOID and with free-form text (all eight settings, Table 12), and action hand-offs cause the most harm on HotpotQA (Table 2).

How harm happens. Harmful trajectories show three patterns: re-checking a trajectory that was already correct, late advice that overwrites correct results (late clarification can be worse than none; [Gulati et al., 2026](#)), and extra retrieval that adds wrong candidate answers. In task 87c610df, the executor alone answers *Morarji Desai* correctly; the advisor sends a coherent message built on a wrong geographic link (Susa → Iran), and the executor switches to *Amir-Abbas Hoveyda*. The advice was not irrational: it rested on a fact the two agents did not share.

We call this the **Compliance Paradox**: the readiness to follow advice that makes rescue possible also carries wrong advice into the answer. Defiance is known to hurt in learning to defer and human–AI teams ([Madras et al., 2018](#); [Bansal et al., 2021](#)); we add which message properties go with more harm (no prohibition, unstructured text, actions instead of explanations). These are recurring patterns, not a causal account. Since a stronger speaker adds rescues but not fewer harms, reducing harm will likely also need listener-side safeguards, which we leave for future work.

8 Conclusion

Our results give simple rules for a stronger model that can only advise. Let the evidence decide who opens the channel: the executor’s own request when failures are silent, a combined error-and-confidence trigger when they are visible. A stronger speaker helps, including across the model families we tested; self-advice gives no gain. Send labelled acts, not free-form text, actions, or answers. Keep an explicit prohibition; a stronger advisor alone does not reduce harm. Language between agents is thus a control channel whose timing, content, and source can be designed and measured.

Limitations

Repetitions. Only the main comparison (no channel, hybrid, and self-initiated for three pairs) was repeated with three seeds. Other configurations were run once, so differences of a few points between two advised policies are not reliable on their own. We did not measure how many tasks change outcome between two runs without advice, so harm counts are upper bounds. **Scale and coverage.** GAIA validation has 165 tasks, and we use a 300-question HotpotQA subset. The two benchmarks were chosen to contrast visible and silent failures, not for breadth; long-horizon web navigation, code repair, and non-English settings are not covered. **Model pairs.** We test five same-family pairs and three cross-family pairs, with executors and advisors of small to moderate size. Other families and larger capability gaps may behave differently. **Mechanism.** We characterise harm through outcomes and qualitative reading of trajectories. We do not label individual messages at scale, so we cannot say how often harmful advice was factually wrong versus unnecessary. **Hand-designed communication.** Initiation policies and message formats are designed by hand, and we do not compare them with learned routers or learned clarification modules (Lin et al., 2026). Learning when and what to communicate, for example with reinforcement learning (Luo et al., 2025), is a natural next step. **Model versions.** The OpenAI results depend on specific API model versions. We release trajectories, prompts, and configurations.

Ethics Statement

The Compliance Paradox has a safety side: an executor that follows a stronger model’s plausible but wrong advice can silently overwrite correct work. In high-stakes settings, this argues for logging advisor messages, using structured messages with explicit prohibitions, and keeping human oversight over final decisions. GAIA and HotpotQA are public research datasets used under their licenses and contain no personal data collected by us. We used AI assistants for writing support and code; all content was checked by the authors.

References

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kap-

paganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. 2025. [Automix: Automatically mixing language models](#). *Preprint*, arXiv:2310.12963.

Alan Agresti. 2012. *Categorical Data Analysis, 3rd Edition*. Wiley, Hoboken, NJ, USA.

Anthropic. 2026. [The advisor strategy: Give agents an intelligence boost](#). Claude blog, April 9, 2026.

Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the whole exceed its parts? the effect of AI explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.

Lingjiao Chen, Matei Zaharia, and James Zou. 2023. [Frugalgpt: How to use large language models while reducing cost and improving performance](#). *Preprint*, arXiv:2305.05176.

Herbert H. Clark and Susan E. Brennan. 1991. Grounding in communication. In Lauren B. Resnick, John M. Levine, and Stephanie D. Teasley, editors, *Perspectives on Socially Shared Cognition*, pages 127–149. American Psychological Association.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. [Hybrid llm: Cost-efficient and quality-aware query routing](#). *Preprint*, arXiv:2404.14618.

Yijiang River Dong, Tiancheng Hu, Zheng Hui, Caiqi Zhang, Ivan Vulić, Andreea Bobu, and Nigel Collier. 2026. [Value of information: A framework for human-agent communication](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 42879–42896.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*.

Zhuoyun Du, Runze Wang, Huiyu Bai, Zouying Cao, Xiaoyong Zhu, Yu Cheng, Bo Zheng, Wei Chen, and Haochao Ying. 2026. [Enabling agents to communicate entirely in latent space](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. 2016. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*.

715	Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong	Saurav Kadavath, Tom Conerly, Amanda Askell, et al.	770
716	Shen, Yujia Yang, Nan Duan, and Weizhu Chen.	2022. Language models (mostly) know what they	771
717	2024. Critic: Large language models can self-	know. <i>arXiv preprint arXiv:2207.05221</i> .	772
718	correct with tool-interactive critiquing . <i>Preprint</i> ,		
719	arXiv:2305.11738.	Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han,	773
		and Rui Zhang. 2024. When can LLMs actually	774
720	H. Paul Grice. 1975. Logic and conversation. In Peter	correct their own mistakes? a critical survey of self-	775
721	Cole and Jerry L. Morgan, editors, <i>Syntax and Se-</i>	correction of LLMs. <i>Transactions of the Association</i>	776
722	<i>mantics</i> , Vol. 3: <i>Speech Acts</i> , pages 41–58. Academic	<i>for Computational Linguistics</i> , 12.	777
723	Press.		
724	Anmol Gulati, Hariom Gupta, Elias Lumer, Sahil	Javid Lakha, Minlan Yu, and Rana Shahout. 2025.	778
725	Sen, and Vamse Kumar Subbiah. 2026. Ask early,	FASTER, CHEAPER, JUST AS GOOD: COST-	779
726	ask late, ask right: When does clarification timing	AND LATENCY-CONSTRAINED ROUTING FOR	780
727	matter for long-horizon agents? <i>arXiv preprint</i>	LLMs . In <i>Sparsity in LLMs (SLLM): Deep Dive into</i>	781
728	<i>arXiv:2605.07937</i> .	<i>Mixture of Experts, Quantization, Hardware, and</i>	782
		<i>Inference</i> .	783
729	Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang,	Angeliki Lazaridou and Marco Baroni. 2020. Emergent	784
730	Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xi-	multi-agent communication in the deep learning era.	785
731	angliang Zhang. 2024. Large language model based	<i>arXiv preprint arXiv:2006.02419</i> .	786
732	multi-agents: A survey of progress and challenges.		
733	In <i>Proceedings of the Thirty-Third International Joint</i>	Guohao Li, Hasan Abed Al Kader Hammoud, Hani	787
734	<i>Conference on Artificial Intelligence</i> .	Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023.	788
		CAMEL: Communicative agents for “mind” explo-	789
735	Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang,	ration of large language model society. In <i>Advances</i>	790
736	Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and	<i>in Neural Information Processing Systems</i> .	791
737	Yang Liu. 2025. Stabletoolbench: Towards stable		
738	large-scale benchmarking on tool learning of large	Xinzhe Li. 2025. A review of prominent paradigms for	792
739	language models . <i>Preprint</i> , arXiv:2403.07714.	llm-based agents: Tool use, planning (including rag),	793
		and feedback learning. In <i>Proceedings of the 31st in-</i>	794
740	Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al.	<i>ternational conference on computational linguistics</i> ,	795
741	2024. MetaGPT: Meta programming for a multi-	pages 9760–9779.	796
742	agent collaborative framework. In <i>International Con-</i>		
743	<i>ference on Learning Representations</i> .	Zheng Li, Xuyun Zhang, Sheng Lu, Hua Deng, Hao	797
		Tian, and Wanchun Dou. 2025. A cost-aware ap-	798
744	Chen Huang, Yuhao Wu, and Wenxuan Zhang. 2026.	proach for collaborating large language models and	799
745	What should agents say? action-state communica-	small language models. In <i>Proceedings of the 34th</i>	800
746	tion for efficient multi-agent systems. <i>arXiv preprint</i>	<i>ACM International Conference on Information and</i>	801
747	<i>arXiv:2606.05304</i> .	<i>Knowledge Management</i> , pages 1748–1757.	802
748	Jie Huang, Xinyun Chen, Swaroop Mishra,	Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang,	803
749	Huaxiu Steven Zheng, Adams Wei Yu, Xiny-	Yan Wang, Rui Wang, Yujia Yang, Shuming Shi, and	804
750	ing Song, and Denny Zhou. 2024a. Large language	Zhaopeng Tu. 2024. Encouraging divergent thinking	805
751	models cannot self-correct reasoning yet. In <i>Interna-</i>	in large language models through multi-agent debate.	806
752	<i>tional Conference on Learning Representations</i> .	In <i>Proceedings of the 2024 Conference on Empirical</i>	807
		<i>Methods in Natural Language Processing</i> .	808
753	Shiting Huang, Zhen Fang, Zehui Chen, Siyu Yuan, Jun-		
754	jie Ye, Yu Zeng, Lin Chen, Qi Mao, and Feng Zhao.	Bohan Lin, Kuo Yang, Zelin Tan, Yingchuan Lai, Chen	809
755	2025. Critictool: Evaluating self-critique capabilities	Zhang, Guibin Zhang, Xinlei Yu, Miao Yu, Xu Wang,	810
756	of large language models in tool-calling error sce-	Yudong Zhang, and Yang Wang. 2026. AgentAsk:	811
757	narios. In <i>Proceedings of the 2025 Conference on</i>	Multi-agent systems need to ask . In <i>Proceedings</i>	812
758	<i>Empirical Methods in Natural Language Processing</i> ,	<i>of the 64th Annual Meeting of the Association for</i>	813
759	pages 26683–26692.	<i>Computational Linguistics (Volume 1: Long Papers)</i> ,	814
		pages 28055–28077.	815
760	Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei	Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang,	816
761	Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruim-	Siyun Zhao, Dongsheng Li, Luna K. Qiu, and	817
762	ing Tang, and Enhong Chen. 2024b. Understanding	Yuqing Yang. 2025. Agent lightning: Train any	818
763	the planning of llm agents: A survey. <i>arXiv preprint</i>	ai agents with reinforcement learning . <i>Preprint</i> ,	819
764	<i>arXiv:2402.02716</i> .	arXiv:2508.03680.	820
765	Mert Inan, Malihe Alikhani, and Anthony Sicilia. 2026.	Aman Madaan, Niket Tandon, Prakhar Gupta, et al.	821
766	Dialogue is the plan: From interface to joint action in	2023. Self-refine: Iterative refinement with self-	822
767	agentic AI . In <i>Proceedings of the 64th Annual Meet-</i>	feedback. In <i>Advances in Neural Information Pro-</i>	823
768	<i>ing of the Association for Computational Linguistics</i>	<i>cessing Systems</i> .	824
769	<i>(Volume 2: Short Papers)</i> , pages 770–779.		

825	David Madras, Toniann Pitassi, and Richard Zemel.	<i>NeurIPS 2023 foundation models for decision making</i>	880
826	2018. Predict responsibly: Improving fairness and	<i>workshop.</i>	881
827	accuracy by learning to defer. In <i>Advances in Neural</i>		
828	<i>Information Processing Systems.</i>		
829	Tula Masterman, Sandi Besen, Mason Sawtell, and Alex	Swarnadeep Saha, Peter Hase, and Mohit Bansal. 2023.	882
830	Chao. 2024. The landscape of emerging ai agent	Can language models teach weaker agents? teacher	883
831	architectures for reasoning, planning, and tool calling:	explanations improve students via personalization. In	884
832	A survey. <i>arXiv preprint arXiv:2404.11584.</i>	<i>Advances in Neural Information Processing Systems.</i>	885
833	Grégoire Mialon, Clémentine Fourrier, Craig Swift,	Maarten Sap, Ronan Le Bras, Daniel Fried, and Yejin	886
834	Thomas Wolf, Yann LeCun, and Thomas Scialom.	Choi. 2022. Neural theory-of-mind? on the limits	887
835	2023. <i>Gaia: a benchmark for general ai assistants.</i>	of social intelligence in large LMs. In <i>Proceedings</i>	888
836	<i>Preprint</i> , arXiv:2311.12983.	<i>of the 2022 Conference on Empirical Methods in</i>	889
837	Sabrina J. Mielke, Arthur Szlam, Emily Dinan, and Y-	<i>Natural Language Processing.</i>	890
838	Lan Boureau. 2022. Reducing conversational agents’		
839	overconfidence through linguistic calibration. <i>Trans-</i>	Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al.	891
840	<i>actions of the Association for Computational Linguis-</i>	2023. Toolformer: Language models can teach them-	892
841	<i>tics</i> , 10.	selves to use tools. In <i>Advances in Neural Informa-</i>	893
842	Hussein Mozannar and David Sontag. 2020. Consis-	<i>tion Processing Systems.</i>	894
843	tent estimators for learning to defer to an expert. In	John R. Searle. 1976. A classification of illocutionary	895
844	<i>Proceedings of the 37th International Conference on</i>	acts. <i>Language in Society</i> , 5(1):1–23.	896
845	<i>Machine Learning.</i>		
846	Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chi-	Mrinank Sharma, Meg Tong, Tomasz Korbak, et al.	897
847	ang, Tianhao Wu, Joseph E. Gonzalez, M Waleed	2024. Towards understanding sycophancy in lan-	898
848	Kadous, and Ion Stoica. 2025. <i>Routellm: Learn-</i>	guage models. In <i>International Conference on Learn-</i>	899
849	<i>ing to route llms with preference data.</i> <i>Preprint</i> ,	<i>ing Representations.</i>	900
850	arXiv:2406.18665.		
851	Ethan Perez, Sam Ringer, Kamile Lukosiute, et al. 2023.	Weizhou Shen, Chenliang Li, Hongzhan Chen, et al.	901
852	Discovering language model behaviors with model-	2024. Small LLMs are weak tool learners: A multi-	902
853	written evaluations. In <i>Findings of the Association</i>	LLM agent. In <i>Proceedings of the 2024 Conference</i>	903
854	<i>for Computational Linguistics: ACL 2023.</i>	<i>on Empirical Methods in Natural Language Process-</i>	904
855	Chau Pham, Boyi Liu, Yingxiang Yang, et al. 2024. Let	<i>ing.</i>	905
856	models speak ciphers: Multiagent debate through em-	Noah Shinn, Federico Cassano, Ashwin Gopinath,	906
857	beddings. In <i>International Conference on Learning</i>	Karthik Narasimhan, and Shunyu Yao. 2023. Re-	907
858	<i>Representations.</i>	flexion: Language agents with verbal reinforcement	908
859	Archiki Prasad, Alexander Koller, Mareike Hartmann,	learning. In <i>Advances in Neural Information Pro-</i>	909
860	Peter Clark, Ashish Sabharwal, Mohit Bansal, and	<i>cessing Systems.</i>	910
861	Tushar Khot. 2024. ADaPT: As-needed decompo-	Madhulika Srikumar, Jacob Pratt, Kasia Chmielin-	911
862	sition and planning with language models. In <i>Find-</i>	ski, Carolyn Ashurst, Chloé Bakalar, William	912
863	<i>ings of the Association for Computational Linguistics:</i>	Bartholomew, Rishi Bommasani, Peter Cihon, Re-	913
864	<i>NAACL 2024.</i>	becca Crotoft, Mia Hoffmann, et al. 2025. Prioritiz-	914
865	Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan	ing real-time failure detection in ai agents. <i>Partner-</i>	915
866	Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang,	<i>ship on AI.</i>	916
867	Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian,	Jimin Sun, So Yeon Min, Yingshan Chang, and Yonatan	917
868	Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li,	Bisk. 2024. <i>Tools fail: Detecting silent errors in</i>	918
869	Zhiyuan Liu, and Maosong Sun. 2023. <i>Toolllm: Fa-</i>	<i>faulty tools.</i> In <i>Proceedings of the 2024 Conference</i>	919
870	<i>cilitating large language models to master 16000+</i>	<i>on Empirical Methods in Natural Language Process-</i>	920
871	<i>real-world apis.</i> <i>Preprint</i> , arXiv:2307.16789.	<i>ing</i> , pages 14272–14289, Miami, Florida, USA. As-	921
872	Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, et al.	sociation for Computational Linguistics.	922
873	2023. Robots that ask for help: Uncertainty align-	Katherine Tian, Eric Mitchell, Allan Zhou, Archit	923
874	ment for large language model planners. In <i>Proceed-</i>	Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn,	924
875	<i>ings of the 7th Conference on Robot Learning.</i>	and Christopher D. Manning. 2023. Just ask for cali-	925
876	Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu,	bration: Strategies for eliciting calibrated confidence	926
877	Tianpeng Bao, Hangyu Mao, Ziyue Li, Xingyu Zeng,	scores from language models fine-tuned with human	927
878	Rui Zhao, et al. 2023. Tptu: Task planning and tool	feedback. In <i>Proceedings of the 2023 Conference on</i>	928
879	usage of large language model-based ai agents. In	<i>Empirical Methods in Natural Language Processing.</i>	929
		Boshi Wang, Xiang Yue, and Huan Sun. 2023. Can	930
		ChatGPT defend its belief in truth? evaluating LLM	931
		reasoning via debate. In <i>Findings of the Association</i>	932
		<i>for Computational Linguistics: EMNLP 2023.</i>	933

934	Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2024. Mixture-of-agents enhances large language model capabilities. <i>arXiv preprint arXiv:2406.04692</i> .	990	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models . <i>Preprint</i> , arXiv:2210.03629.	991
935		992		993
936				
937				
938	Ningning Wang, Xavier Hu, Pai Liu, He Zhu, Yue Hou, Heyuan Huang, Shengyu Zhang, Jian Yang, Jiaheng Liu, Ge Zhang, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. 2025a. Efficient agents: Building effective agents while reducing cost . <i>Preprint</i> , arXiv:2508.02694.	994	Yuxiao Ye, Yiming Zhang, Yiran Rex Ma, Huiyuan Xie, Huining Zhu, and Zhiyuan Liu. 2026. LinguaGame: A linguistically grounded game-theoretic paradigm for multi-agent dialogue generation. In <i>Findings of the Association for Computational Linguistics: ACL 2026</i> , pages 20544–20558.	995
939		996		997
940		998		999
941				
942				
943				
944	Rongzheng Wang, Yihong Huang, Muquan Li, Jiakai Li, Di Liang, Bob Simons, Pei Ke, Shuang Liang, and Ke Qin. 2026. Rethinking llm-driven heuristic design: Generating efficient and specialized solvers via dynamics-aware optimization . <i>Preprint</i> , arXiv:2601.20868.	1000	Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. 2024. Large language model cascades with mixture of thought representations for cost-efficient reasoning. In <i>International Conference on Learning Representations</i> .	1001
945		1002		1003
946		1004		
947				
948				
949				
950	Xinyuan Wang, Yanchi Liu, Wei Cheng, Xujiang Zhao, Zhengzhang Chen, Wenchao Yu, Yanjie Fu, and Haifeng Chen. 2025b. Mixllm: Dynamic routing in mixed large language models . <i>Preprint</i> , arXiv:2502.18482.	1005	Yunxiang Zhang, Muhammad Khalifa, Lajanugen Logeswaran, et al. 2024. Small language models need strong verifiers to self-correct reasoning. In <i>Findings of the Association for Computational Linguistics: ACL 2024</i> .	1006
951		1007		1008
952		1009		
953				
954				
955	Robert West, Ashton Anderson, Ece Kamar, and Eric Horvitz. 2026. Tandem training for language models. In <i>Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)</i> .	1010	Yuzhe Zhang, Feiran Liu, Yi Shan, Xinyi Huang, Xin Yang, Yueqi Zhu, Xuxin Cheng, Cao Liu, Ke Zeng, Terry Jingchen Zhang, and Wenyuan Jiang. 2026. SILO-BENCH: A scalable environment for evaluating distributed coordination in multi-agent LLM systems . In <i>Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> .	1011
956		1012		1013
957		1014		1015
958		1016		1017
959				
960	Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. <i>arXiv preprint arXiv:2308.08155</i> .	1018	Kaitlyn Zhou, Jena D. Hwang, Xiang Ren, and Maarten Sap. 2024. Relying on the unreliable: The impact of language models’ reluctance to express uncertainty. In <i>Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics</i> .	1019
961		1020		1021
962		1022		
963				
964				
965				
966				
967	Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2024. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In <i>International Conference on Learning Representations</i> .	1023	A Implementation Details	
968		1024	Agent loop. Algorithm 1 shows the loop shared by all conditions. The executor acts step by step. After each tool call, the initiation policy decides whether to consult the advisor, and the advisor’s message is added to the executor’s context before its next step. If the step limit is reached without an answer, a single forced-answer prompt makes the executor commit, so every run ends with an answer.	1025
969		1026		1027
970		1028		1029
971		1030		1031
972	Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In <i>International Conference on Learning Representations</i> .	1032		
973				
974				
975				
976				
977	Xiaofang Yang, Lijun Li, Heng Zhou, Tong Zhu, Xiaoye Qu, Yuchen Fan, Qianshan Wei, Rui Ye, Li Kang, Yiran Qin, Zhiqiang Kou, Daizong Liu, Qi Li, Ning Ding, Siheng Chen, and Jing Shao. 2026. Toward efficient agents: Memory, tool learning, and planning . <i>Preprint</i> , arXiv:2601.14192.	1033	Budgets and decoding. Step limit $T = 16$; at most 12 tool calls and $B = 2$ advisor messages per task. Report policies use $\tau \in \{0.25, 0.5, 0.75\}$; the hybrid uses $\tau = 0.75$. GPT-4.1 models use temperature 0; GPT-5.4 models use low reasoning effort. Seed 42, plus seeds 43 and 44 for the repeated runs (Table 7). Qwen and gpt-oss models are served with vLLM on NVIDIA RTX 6000 Ada GPUs (48 GB); decoding settings are included in the released configuration files.	1034
978		1035		1036
979		1037		1038
980		1039		1040
981		1041		1042
982				
983	Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In <i>Proceedings of the 2018 conference on empirical methods in natural language processing</i> , pages 2369–2380.			
984				
985				
986				
987				
988				
989				

Algorithm 1 Advisor Pattern execution loop

Require: Task x , executor M_e , advisor M_a , initiation policy π , message format μ , step limit T , message budget B

```
1:  $s_0 \leftarrow \text{INIT}(x)$ ;  $b \leftarrow 0$ 
2: for  $t = 0$  to  $T - 1$  do
3:    $a_t \leftarrow M_e(s_t)$  {executor acts}
4:   if  $a_t$  is a final answer then
5:     return  $a_t$ 
6:   end if
7:    $o_t \leftarrow \text{EXECUTETOOL}(a_t)$ 
8:    $s_{t+1} \leftarrow \text{UPDATE}(s_t, a_t, o_t)$ 
9:   if  $\pi(s_{t+1}) = 1$  and  $b < B$  then
10:     $g \leftarrow M_a(\text{CONTEXT}(x, s_{t+1}); \mu)$  {advisor message}
11:     $s_{t+1} \leftarrow \text{ADD}(s_{t+1}, g)$ ;  $b \leftarrow b + 1$ 
12:   end if
13: end for
14: return  $M_e(s_T \parallel \text{FORCEANSWER})$  {step limit reached}
```

Policies. The monitor fires on tool exceptions, malformed tool outputs, and repeated identical calls. The report policy asks the executor for a confidence between 0 and 1 after each step. The self-initiated policy gives the executor a help-request action that it can choose instead of a tool call. Random initiation opens the channel at random steps with the same message budget.

Prices. Table 4 lists the prices used for all cost figures. Open-weight models are served locally, so we report GPU time instead of dollar cost.

Model	Input (\$/1M)	Output (\$/1M)
GPT-5.4 Nano	0.20	1.25
GPT-4.1 Mini	0.40	1.60
GPT-4.1	2.00	8.00
GPT-5.4 (advisor)	2.50	15.00

Qwen pair: 0.45 GPU-hours per 100 tasks (local vLLM)
gpt-oss pair: 0.65 GPU-hours per 100 tasks (local vLLM)

Table 4: Prices used for cost figures (OpenAI list prices, September 30, 2026) and GPU time for the open-weight pairs.

Code and data. Code, prompts, configurations, and all trajectory logs: <https://anonymous.4open.science/r/advisor-eval>.

B Additional Results

Easy and hard tasks. On easy GAIA tasks, advice fully makes up for a weaker executor: with the hybrid policy, every OpenAI pair reaches 98%, the same as GPT-5.4 alone (Table 9). For the open-weight pairs, advice closes most of the easy-task gap (Qwen 57% $\rightarrow$ 77%, vs. 81% for its advisor alone; OSS 64% $\rightarrow$ 81%, vs. 89%). On hard tasks,

advice helps a lot (e.g., 4% $\rightarrow$ 24% for Nano) but stays below the advisor alone in every pair. Short advice has limits when the task itself requires long chains of tool use.

Reliability. For every pair, the hybrid produces the same or fewer tool errors than the best single-signal policy (Table 10). The open-weight pairs gain fewer accuracy points per advisor message (0.08–0.11) than the OpenAI pairs (0.14–0.20).

C Case Studies

How helpful messages work on GAIA. Helpful messages mostly repair tool use. Weaker executors make more tool errors, receive more messages, and gain the most (Table 10). Helpful messages fall into four patterns. (i) *Fixing tool inputs*: encoding spaces as %20, replacing made-up endpoints, or reading the unread sheets of an XLSX file instead of empty rendered cells. (ii) *Breaking loops*: AVOID names the failing URL and NEXT gives a working one, which stops repeated 404 errors. (iii) *Naming a missing source*: for example, adding a lookup of Chinese premiers when the task requires intersecting them with OpenCV contributors. (iv) *Fixing the answer format*: AVOID names a counting or formatting error and NEXT states the required output format. Figure 4 (task f46b4380) shows a rescue that combines patterns (i) and (iv). Section 7 describes a harmful case (task 87c610df).

Pair	Policy	Acc.	Cost red.	Lat. (s)	Pair	Policy	Acc.	Cost red.	Lat. (s)
Nano	No channel	20	97.5	7.46	4.1	No channel	31	22.7	14.86
	Self-initiated	40	78.8	16.89		Self-initiated	46	16.9	17.97
	Monitor	39	80.2	15.81		Monitor	43	17.7	17.15
	Random	18	75.0	17.52		Random	36	−37.1	19.99
	Report $\tau=.25$	41	75.8	17.53		Report $\tau=.25$	42	−28.1	18.68
	Report $\tau=.5$	43	74.5	16.63		Report $\tau=.5$	45	−25.3	18.51
	Report $\tau=.75$	45	73.0	17.12		Report $\tau=.75$	48	−18.7	18.83
	Hybrid	48	73.5	15.74		Hybrid	50	1.4	17.90
Mini	No channel	26	91.3	12.69	Qwen	No channel	23	−	6.43
	Self-initiated	46	80.1	18.29		Self-initiated	35	−	6.98
	Monitor	42	80.9	18.77		Monitor	31	−	7.01
	Random	31	66.1	22.02		Random	27	−	6.95
	Report $\tau=.25$	42	74.7	17.83		Report $\tau=.75$	35	−	6.99
	Report $\tau=.5$	45	75.9	18.04		Hybrid	36	−	7.03
	Report $\tau=.75$	46	67.3	18.71	OSS	No channel	27	−	7.94
	Hybrid	48	74.3	18.78		Self-initiated	37	−	8.24
Advisor alone: GPT-5.4 55% (17.91 s); Qwen3.6-27B 44% (8.48 s); gpt-oss-120b 49% (10.34 s).						Monitor	31	−	8.45
						Random	30	−	8.43
						Report $\tau=.75$	38	−	8.35
						Hybrid	38	−	8.47

Table 5: All initiation policies on GAIA (165 tasks). Accuracy in %. Cost reduction is relative to running GPT-5.4 alone; it is not reported for the locally served open-weight pairs (see Table 4).

Policy	Nano	Mini	4.1	Qwen	OSS
No channel	61	69	73	61	64
Random	65 (34/22)	71 (22/16)	78 (32/13)	64 (28/19)	67 (29/20)
Monitor	79 (66/12)	77 (35/11)	84 (41/9)	70 (37/10)	72 (34/10)
Report $\tau=.5$	79 (65/11)	80 (44/11)	83 (40/8)	71 (38/8)	78 (51/9)
Report $\tau=.75$	80 (66/9)	81 (45/9)	84 (44/8)	75 (50/8)	80 (56/8)
Hybrid	78 (58/7)	82 (49/10)	87 (45/5)	75 (51/9)	80 (57/9)
Self-initiated	82 (67/4)	85 (53/5)	89 (50/3)	77 (54/6)	81 (58/7)
Advisor alone	78	78	78	76	76

Table 6: All initiation policies on HotpotQA (300 questions, seed 42). Cells: accuracy (%) with (help / harm) against the executor alone. Random initiation has the most harm and the lowest help/harm ratio in every pair.

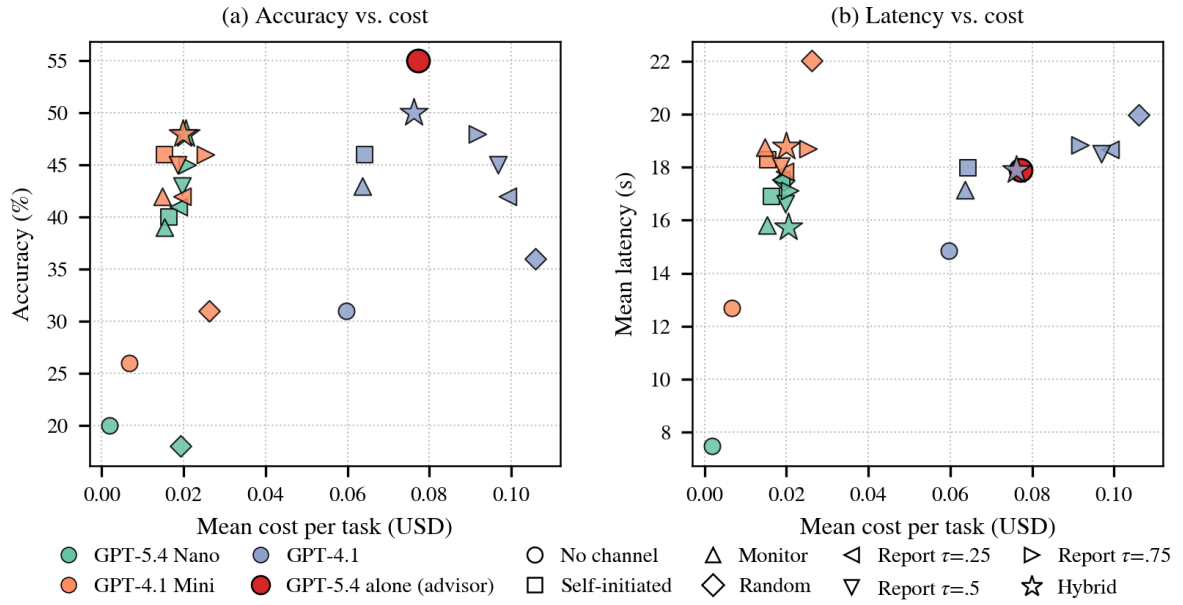


Figure 3: Cost, accuracy, and latency of initiation policies on GAIA for the OpenAI pairs (values from Table 6). (a) Accuracy vs. mean cost per task; (b) mean latency vs. cost. Colour marks the executor, shape the initiation policy; the red point is GPT-5.4 running alone. The Nano and Mini hybrid points nearly coincide (both 48%).

Pair	Bench.	Policy	s42	s43	s44	Range
Nano	GAIA	No channel	20	20	20	0
		Hybrid	48	47	48	1
		Self-initiated	40	41	41	1
	Hotpot	No channel	61	61	62	1
		Hybrid	78	78	77	1
		Self-initiated	82	82	81	1
Qwen	GAIA	No channel	23	23	22	1
		Hybrid	36	36	35	1
		Self-initiated	35	35	36	1
	Hotpot	No channel	61	62	61	1
		Hybrid	75	75	75	0
		Self-initiated	77	77	77	0
OSS	GAIA	No channel	27	26	26	1
		Hybrid	38	36	37	2
		Self-initiated	37	37	35	2
	Hotpot	No channel	64	63	64	1
		Hybrid	80	80	81	1
		Self-initiated	81	82	82	1

Table 7: Accuracy (%) across three seeds for the main comparison. Hybrid minus self-initiated per seed: Nano +8/ +6/ +7 (GAIA) and -4/ -4/ -4 (HotpotQA); Qwen +1/ +1/ -1 and -2/ -2/ -2; OSS +1/ -1/ +2 and -1/ -2/ -1.

Executor → advisor	GAIA		HotpotQA		Cost (\$/task)	
	Hybrid	Self	Hybrid	Self	GAIA	Hotpot
Qwen3.5-9B → GPT-5.4	39	38	80	81	0.0163	0.0099
gpt-oss-20b → GPT-5.4	40	40	81	82	0.0147	0.0087
Nano → Qwen3.6-27B	43	44	76	77	-	-

Table 8: Cross-family pairs: hybrid vs. self-initiated accuracy (%), and API cost of the GPT-5.4 advisor. With GPT-5.4 advising, the advisor sends 161 (Qwen) and 175 (gpt-oss) messages on GAIA, with 152 and 178 tokens per message.

Pair	Configuration	Easy (%)	Hard (%)
Nano	No channel	53 (28/53)	4 (5/112)
	Best single (Report $\tau=.75$)	94 (50/53)	21 (24/112)
	Hybrid	98 (52/53)	24 (27/112)
Mini	No channel	58 (31/53)	10 (12/112)
	Best single (Self-initiated)	96 (51/53)	23 (26/112)
	Hybrid	98 (52/53)	24 (27/112)
4.1	No channel	75 (40/53)	10 (12/112)
	Best single (Report $\tau=.75$)	96 (51/53)	25 (28/112)
	Hybrid	98 (52/53)	27 (31/112)
Advisor alone (GPT-5.4)		98 (52/53)	35 (39/112)
Qwen	No channel	57 (30/53)	7 (8/112)
	Best single (Report $\tau=.75$)	77 (41/53)	15 (17/112)
	Hybrid	77 (41/53)	16 (18/112)
	Advisor alone (Qwen3.6-27B)	81 (43/53)	26 (29/112)
OSS	No channel	64 (34/53)	10 (11/112)
	Best single (Report $\tau=.75$)	83 (44/53)	17 (19/112)
	Hybrid	81 (43/53)	18 (20/112)
	Advisor alone (gpt-oss-120b)	89 (47/53)	30 (34/112)

Table 9: GAIA accuracy by difficulty. Easy = Level 1 (53 tasks); Hard = Levels 2 and 3 (112 tasks).

Situation (GAIA f46b4380). The executor fetched a review page with an unencoded space in the URL and is about to return one item. The task asks for a comma-separated list covering several albums.

DIAGNOSE: The fetch failed because the URL has a raw space, so the review was never read. The current answer covers only one album.

AVOID: Do not retry the same URL. Do not answer with a single item.

NEXT: Re-fetch with the space encoded as %20, check each album against the criterion, and return all matches, comma-separated.

Outcome: The executor finds the missing album and answers *Harbinger*, *Tidal* (correct). The advisor never states the answer.

Figure 4: A DNA message that rescued a failing trajectory (abridged and paraphrased from the log).

Pair	Policy	Accuracy gain	Tool errors	Advisor messages	Messages after an error	Recovery efficiency
Nano	Report $\tau=.75$	+25	58	176	57	0.142
Nano	Hybrid	+28	57	189	47	0.148
Mini	Self-initiated	+20	47	112	34	0.179
Mini	Hybrid	+22	36	135	32	0.163
4.1	Report $\tau=.75$	+17	34	97	16	0.175
4.1	Hybrid	+19	33	96	17	0.198
Qwen	Report $\tau=.75$	+12	67	152	52	0.079
Qwen	Hybrid	+13	60	142	47	0.092
OSS	Report $\tau=.75$	+11	52	112	39	0.098
OSS	Hybrid	+11	49	101	42	0.109

Table 10: Execution reliability on GAIA (totals over 165 tasks). Recovery efficiency = accuracy points gained per advisor message.

Pair	Policy	Harm	Harm %	Help	Help %	Net	Help/Harm
Nano	Report $\tau=.75$	5	3.0	46	27.9	+41	9.2×
Mini	Self-initiated	4	2.4	37	22.4	+33	9.3×
4.1	Report $\tau=.75$	6	3.6	34	20.6	+28	5.7×
Qwen	Report $\tau=.75$	10	6.1	30	18.2	+20	3.0×
OSS	Report $\tau=.75$	8	4.8	26	15.8	+18	3.3×
Nano	Hybrid	7	4.2	53	32.1	+46	7.6×
Mini	Hybrid	5	3.0	41	24.8	+36	8.2×
4.1	Hybrid	6	3.6	37	22.4	+31	6.2×
Qwen	Hybrid	9	5.5	30	18.2	+21	3.3×
OSS	Hybrid	8	4.8	26	15.8	+18	3.3×

Table 11: Help and harm on GAIA (165 tasks) against the executor alone. Top: best single-signal policy per pair. Bottom: hybrid. Every row improves significantly over the executor alone (exact McNemar test, $p \leq 0.003$).

Message	Nano	Mini	GAIA 4.1	Qwen	OSS	Nano	HotpotQA Qwen	OSS
No channel	20 / –	26 / –	31 / –	23 / –	27 / –	61 / –	61 / –	64 / –
Full DNA (DIAGNOSE + AVOID + NEXT)	48 / 7	48 / 5	50 / 6	36 / 9	38 / 8	78 / 7	75 / 9	80 / 9
DIAGNOSE + NEXT (no AVOID)	45 / 10	46 / 9	47 / 8	35 / 11	35 / 10	75 / 10	72 / 12	78 / 11
AVOID + NEXT (no DIAGNOSE)	39 / 11	43 / 8	48 / 7	33 / 9	33 / 9	74 / 9	71 / 11	77 / 10
DIAGNOSE + AVOID (no NEXT)	37 / 8	40 / 7	46 / 7	30 / 10	33 / 9	74 / 8	69 / 10	76 / 10
NEXT only	34 / 9	41 / 9	43 / 10	32 / 13	33 / 11	73 / 10	71 / 11	75 / 10
DIAGNOSE only	31 / 6	35 / 8	40 / 7	29 / 13	30 / 11	70 / 11	70 / 13	73 / 11
AVOID only	25 / 4	34 / 5	37 / 6	26 / 10	31 / 9	69 / 8	69 / 10	72 / 10
Free-form (no labelled acts)	35 / 11	40 / 10	41 / 10	29 / 14	30 / 11	67 / 10	69 / 11	72 / 10

Table 12: Which parts of the message matter. Each cell shows accuracy (%) / number of harm events (tasks that were right without advice and wrong with it). Hybrid initiation; only the message changes. Bold: best per column. Pairs as in Table 1.

Message	Acc. (%)	Acc. gain	Cost red. (%)	Tool errors	Advisor messages	Help	Harm
No channel	20	–	97.5	–	0	–	–
Full DNA	48	+28	73.5	57	189	53	7
DIAGNOSE + NEXT	45	+25	70.5	65	196	53	10
AVOID + NEXT	39	+19	69.7	67	201	47	11
DIAGNOSE + AVOID	37	+17	67.1	88	246	43	8
Free-form	35	+15	62.3	68	223	41	11
NEXT only	34	+14	62.3	62	201	49	9
DIAGNOSE only	31	+11	60.8	69	220	46	6
AVOID only	25	+5	51.6	75	256	39	4

Table 13: Full message ablation for Nano on GAIA (hybrid initiation, 165 tasks). Without NEXT, the executor makes the most tool errors (88).